

# VehicleHierarchy Report

## 1. Design Choices and Inheritance Structure

The program models a vehicle hierarchy using a single Java file, `VehicleHierarchy.java`, with an abstract superclass `Vehicle` and two concrete subclasses: `Car` and `Motorcycle`.

### Superclass (Vehicle)

The `Vehicle` class serves as the base of the hierarchy. It encapsulates three core attributes shared by all vehicles: `make`, `model`, and `year`. These fields are declared as private to enforce encapsulation, ensuring they can only be accessed or modified through the class's own methods. A constructor initializes these attributes, and a `displayInfo()` method prints the vehicle's details in a formatted string.

The class is declared abstract because a generic "Vehicle" on its own is too broad to instantiate meaningfully. Every vehicle in the real world is a specific type, so making `Vehicle` abstract reflects this by requiring instantiation through a subclass.

### Subclasses (Car and Motorcycle)

Both subclasses extend `Vehicle`, inheriting its fields and methods. Each adds a unique attribute:

- `Car` adds `numDoors` (an int representing the number of doors).
- `Motorcycle` adds `hasSidecar` (a boolean indicating sidecar presence).

Both subclasses use the `super` keyword in their constructors to delegate initialization of the inherited fields (`make`, `model`, `year`) to the `Vehicle` constructor. This avoids code duplication and ensures consistent initialization logic.

## 2. Method Overriding

Both `Car` and `Motorcycle` override the `displayInfo()` method from `Vehicle`. The overriding strategy follows two steps:

- Call `super.displayInfo()` to print the common vehicle information.
- Print the subclass-specific attribute (`numDoors` or `hasSidecar`).

This approach avoids duplicating the base display logic in each subclass. If the format of the common fields ever changes, only the `Vehicle` class needs to be updated. The `@Override` annotation is used on each overriding method, which allows the compiler to verify that the method signature matches the parent class.

## 3. Polymorphism

In the `main()` method, an array of type `Vehicle[]` stores instances of both `Car` and `Motorcycle`. A single for loop iterates through the array and calls `displayInfo()` on each element. At runtime, Java determines the actual type of each object and calls the appropriate overridden method. This is polymorphism: the same method call produces different output depending on the object's actual type.

This demonstrates the power of polymorphism, where new vehicle types can be added in the future without modifying the loop or any existing code that operates on `Vehicle` references.

## **4. Challenges Faced and Solutions Applied**

### **Inner Classes vs. Top-Level Classes**

Since the assignment requires a single `.java` file, all classes had to be defined inside `VehicleHierarchy`. Initially, the subclasses were non-static inner classes, which required creating an instance of the outer class to instantiate them. This was resolved by declaring the inner classes as static, which removed the dependency on the outer class instance and simplified the instantiation syntax.

### **Deciding Whether Vehicle Should Be Abstract**

The decision to make `Vehicle` abstract required changing the `main()` method so that no plain `Vehicle` objects were instantiated. This change better models the real-world concept where a vehicle is always a specific type.

## **5. Benefits of Inheritance and Polymorphism**

### **Code Reuse**

Inheritance allows `Car` and `Motorcycle` to share the common fields and constructor logic defined in `Vehicle`. Without inheritance, these would need to be duplicated in each class, increasing maintenance effort and the risk of inconsistencies.

### **Flexibility and Extensibility**

Polymorphism allows code to operate on `Vehicle` references without knowing the specific subclass. Adding a new vehicle type requires only creating a new subclass, with no additional changes to existing code that uses `Vehicle` references.

### **Maintainability**

Changes to shared behavior only need to happen in one place: the `Vehicle` superclass. Subclasses automatically inherit the updated behavior through `super.displayInfo()`.